Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores 2.
Ing. Mario Tobar.

“MANUAL DE USUARIO - ST4CK OVER-PIG”

Diego José Maldonado Monterroso.
Carné: 201931811.

Quetzaltenango, Guatemala.

Martes 27 de Octubre de 2026.

Introducción

St4ck Over-Pig es un editor y compilador para tres lenguajes: **PigLatin**, **Y?** y **Zetariano**. Usted escribe su programa, lo compila, revisa sus errores, ve cómo se traduce y lo **ejecuta dentro de la misma aplicación**, sin usar una terminal. El resultado es un programa en ensamblador **RISC-V** que también puede comprobar por fuera, en el simulador RARS.

Este manual explica, paso a paso y con capturas de pantalla, cómo instalar la aplicación y cómo usar cada una de sus funciones. Está pensado para quien escribe y prueba programas; los detalles internos están en el manual técnico.

En este manual verá cómo instalar y abrir la aplicación; conocer sus botones y las partes de la ventana; abrir, crear, guardar y descargar archivos y carpetas; escribir en los tres lenguajes; compilar y corregir los errores; ver el AST, los símbolos, la pila, las cuartetas, el código 3D y el RISC-V; ejecutar el programa; comprobar el ensamblador en RARS; probar los ejemplos incluidos y resolver los problemas frecuentes.

Requisitos e instalación

Lo que necesita

- **Java (JDK) 21 o superior.** Para saber si lo tiene, abra una terminal y escriba `java -version`.
- **Maven 3.9 o superior.** Compruébelo con `mvn -v`.
- Windows 10/11, Linux o macOS, y una pantalla de al menos 1366 × 768.
- Conexión a Internet la primera vez, para que Maven descargue lo que la aplicación usa.

No hace falta instalar RARS: viene dentro del proyecto, en la carpeta `lib/`.

Obtener y abrir la aplicación

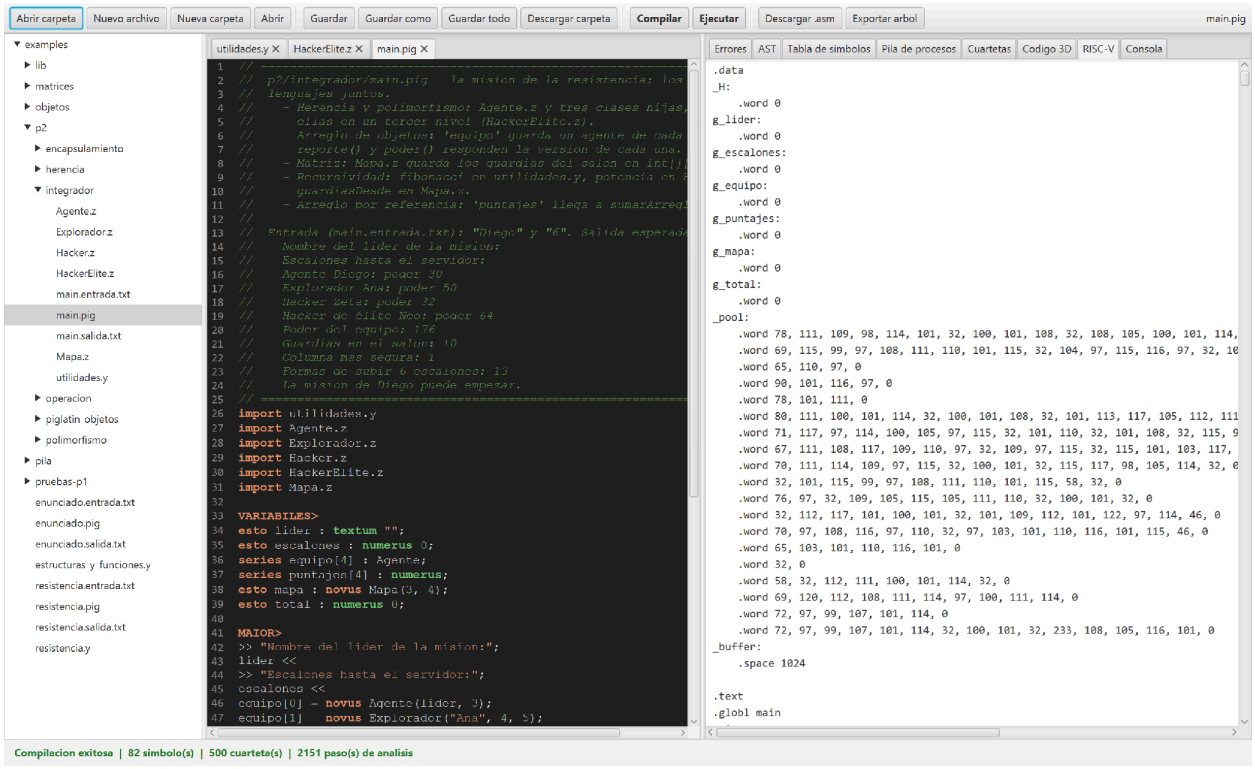
1. Descargue el proyecto como ZIP desde GitHub y descomprímalo, o clónelo con:

   ```
   git clone https://github.com/DiegoMaldonado19/st4ck-over-pig.git
   ```

2. Abra una terminal dentro de la carpeta `st4ck-over-pig`.
3. Escriba `mvn clean javafx:run` y presione Enter.
4. La primera vez Maven tarda un poco más, porque descarga sus dependencias. Después se abre la ventana de St4ck Over-Pig.

Con NetBeans: abra la carpeta como proyecto Maven (*File → Open Project*) y presione *Run*. El proyecto ya trae la configuración para abrir la aplicación.

La ventana principal



La ventana con la carpeta examples abierta y un programa compilado.

La ventana tiene cinco partes:

1. **Barra de herramientas** (arriba): los botones de archivos, compilación, ejecución y descargas. A la derecha se ve el nombre del archivo activo.
2. **Árbol de trabajo** (izquierda): las carpetas y archivos de la carpeta abierta.
3. **Editores** (centro): una pestaña por archivo abierto, con el código coloreado. Un `*` antes del nombre indica cambios sin guardar.
4. **Resultados** (derecha): las pestañas Errores, AST, Tabla de simbolos, Pila de procesos, Cuartetas, Codigo 3D, RISC-V y Consola.
5. **Barra de estado** (abajo): el resultado de la última acción, en verde si salió bien y en rojo si hubo errores.

Los separadores entre las partes se pueden arrastrar para dar más espacio a una de ellas.



La barra de herramientas.

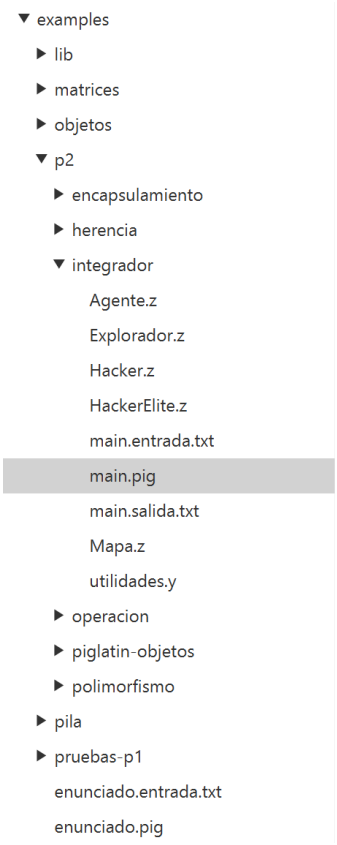
Los botones de la barra de herramientas.

Botón	**Qué hace**
Abrir carpeta	Elige la carpeta de trabajo y la muestra en el árbol.
Nuevo archivo	Con una carpeta abierta, pide un nombre y crea el archivo donde está la selección del árbol. Sin carpeta, abre una pestaña "Sin titulo".
Nueva carpeta	Pide un nombre y crea una carpeta donde está la selección del árbol.
Abrir	Abre un archivo `.pig`, `.y` o `.z` y lo compila de una vez.
Guardar	Guarda el archivo activo. Si nunca se guardó, pregunta dónde.
Guardar como	Guarda una copia con otro nombre o en otro lugar. También sirve para descargar un archivo.
Guardar todo	Guarda todos los archivos con cambios.
Descargar carpeta	Copia la carpeta de trabajo completa, con sus subcarpetas, al lugar que usted elija.
Compilar	Analiza el archivo activo y muestra sus resultados.
Ejecutar	Compila el archivo activo y ejecuta su ensamblador dentro de la aplicación.
Descargar .asm	Guarda el ensamblador RISC-V junto al archivo, con el mismo nombre: `main.pig` → `main.asm`.
Exportar arbol	Guarda el AST como texto (`.txt`).

Archivos y carpetas

Abrir una carpeta de trabajo

1. Pulse "Abrir carpeta" y elija una carpeta, por ejemplo `examples` del proyecto.
2. El árbol muestra primero las carpetas y después los archivos, por nombre. Pulse el triángulo de una carpeta para abrirla.
3. Haga **doble clic** en un archivo para abrirlo en una pestaña.



El árbol de trabajo con p2/integrador abierto.

Crear archivos y carpetas

1. En el árbol, seleccione la carpeta donde quiere crear (o un archivo de esa carpeta).
2. Pulse "Nuevo archivo" o "Nueva carpeta".
3. Escriba el nombre, con su extensión si es un archivo (`nuevo.pig`, `Persona.z`, `utilidades.y`), y acepte.

Si todavía no abrió una carpeta, "Nuevo archivo" abre una pestaña en blanco ("Sin titulo"). La primera vez que la guarde, la aplicación le pedirá el nombre y el lugar.

Guardar y descargar

- "Guardar" guarda el archivo activo; "Guardar todo", todos los que tienen cambios.
- "Guardar como" guarda una copia en otro lugar o con otro nombre: es la forma de descargar un archivo. La pestaña pasa a llevar el nombre nuevo y se colorea según su extensión.
- "Descargar carpeta" copia la carpeta de trabajo completa dentro de la carpeta que usted elija. No puede elegir una carpeta que esté dentro de la de trabajo.
- Al compilar, la aplicación guarda primero los archivos abiertos que tengan cambios: lo que ve en las pestañas es lo que se compila.

Si cierra una pestaña o la ventana con cambios sin guardar, la aplicación le pregunta antes de perderlos.

Escribir un programa

Un programa empieza siempre en un archivo **PigLatin** (`.pig`), que puede importar funciones y estructuras de **Y?** (`.y`) y clases de **Zetariano** (`.z`).

Los tres lenguajes.

Lenguaje	Extensión	Para qué sirve	Estilo
PigLatin	`.pig`	El programa principal: importaciones, variables globales (`VARIABILES>`), funciones (`MUNERA>`) y el bloque principal (`MAIOR>`), que termina con `FINIS;`.	Palabras en latín y bloques `{ } finis;`
Y?	`.y`	Estructuras (`%estructuras`) y funciones (`%funciones`).	Como Python: los bloques van por sangría.
Zetariano	`.z`	Una clase por archivo, con atributos, métodos, constructores, encapsulamiento y herencia.	Como Java.

Un primer programa con los tres lenguajes

Este es el ejemplo `examples/pruebas-p1/00_hello_world`. El `.pig` importa una función de Y? y una clase de Zetariano:

HolaMundo.pig

```
import Saludo.y
import Hola.z

VARIABILES>
esto objeto : novus Hola();
esto nombre : textum "Cadete";

MAIOR>
>> "Hola Mundo desde Pig Latin" ;
saludar();
saludarA(nombre);
objeto.decir();
>> "2 + 3 = " >> sumar(2, 3) ;
>> "El doble de 21 es " >> objeto.doble(21) ;
FINIS;
```

Saludo.y

```
%funciones

definir saludar():
    imprimir("Hola Mundo desde Y?")

definir saludarA(cadena nombre):
    imprimir("Hola " + nombre)

definir sumar(entero a, entero b) -> entero:
    retornar a + b
```

Hola.z

```
public class Hola {
    String mensaje;

    public Hola() {
        mensaje = "Hola Mundo desde Zetariano";
    }

    public void decir() {
        println(mensaje);
    }

    public int doble(int n) {
        return n * 2;
    }
}
```

Al ejecutarlo imprime:

```
Hola Mundo desde Pig Latin
Hola Mundo desde Y?
Hola Cadete
Hola Mundo desde Zetariano
2 + 3 = 5
El doble de 21 es 42
```

Reglas que conviene recordar

- Un `import` se escribe con la ruta relativa a la carpeta del `.pig`: `import lib.Funciones.y` busca `lib/Funciones.y`. Las mayúsculas cuentan: `Hola.z` no es `hola.z`.
- Un archivo `.z` contiene una sola clase y **se llama como ella**: la clase `Persona` va en `Persona.z`.
- El `.pig` importa **todos** los archivos que usa el programa, también una clase que solo usa otra clase (por ejemplo `Nodo.z`, que usa `Pila.z`). Un `.z` no tiene `import`: compilado solo, ve a las demás clases de su carpeta.
- Guarde el `.pig` antes de compilarlo si tiene importaciones: la aplicación las busca en disco junto a él.
- En Y? la sangría es parte del lenguaje: use siempre la misma cantidad de espacios en un bloque.
- En Zetariano, un atributo, método o constructor sin modificador (default) funciona como `private`: solo se usa dentro de su clase. Si lo usa una clase hija, márquelo `protected`; si lo usa otra clase o el `.pig`, `public`.

- Un método que sobrescribe a uno del padre lleva `@Override` y no puede reducir su visibilidad. Si el padre retorna un objeto, la hija puede retornar una de sus clases hijas.

El coloreado

El editor colorea el código mientras escribe, con el analizador léxico del propio lenguaje: lo que se pinta de rojo es lo que el compilador no reconoce.

Los colores del editor.

Color	**Qué marca**	**Ejemplos**
Naranja, en negrita	Palabras reservadas	`esto`, `si`, `definir`, `class`, `extends`, `@Override`
Verde, en negrita	Tipos y valores lógicos	`numerus`, `entero`, `int`, `verum`, `true`, `null`
Verde claro	Números	`42`, `9.81`
Salmón	Textos y caracteres	`"Hola"`, `'a'`
Verde, en cursiva	Comentarios	`// nota`
Rojo, subrayado	Errores léxicos	un texto sin cerrar, un símbolo desconocido

utilidades.y × | HackerElite.z × | main.pig ×

```
// ============================================================
import utilidades.y
import Agente.z
import Explorador.z
import Hacker.z
import HackerElite.z
import Mapa.z

VARIABILES>
esto lider : textum "";
esto escalones : numerus 0;
series equipo[4] : Agente;
series puntajes[4] : numerus;
esto mapa : novus Mapa(3, 4);
esto total : numerus 0;

MAIOR>
>> "Nombre del lider de la mision:";
lider <<
>> "Escalones hasta el servidor:";
escalones <<
equipo[0] = novus Agente(lider, 3);
equipo[1] = novus Explorador("Ana", 4, 5);
equipo[2] = novus Hacker("Zeta", 5);
equipo[3] = novus HackerElite("Neo");
per (esto i : numerus 0; i < 4; i++) {
    >> equipo[i].reporte();
    puntajes[i] = equipo[i].poder();
}
total = sumarArreglo(puntajes, 4);
>> "Poder del equipo: " >> total;
mapa.vigilar(0, 0, 2);
mapa.vigilar(0, 3, 1);
mapa.vigilar(1, 0, 1);
mapa.vigilar(1, 2, 1);
mapa.vigilar(2, 2, 3);
mapa.vigilar(2, 3, 2);
>> "Guardias en el salon: " >> mapa.guardiasDesde(0);
>> "Columna mas segura: " >> mapa.columnaMasSegura();
>> "Formas de subir " >> escalones >> " escalones: " >> fibona
si (total > mapa.guardiasDesde(0) * 10) {
    >> "La mision de " >> lider >> " puede empezar.";
} aliter {
    >> "Faltan agentes.";
} finis;
FINIS;
```

Un programa PigLatin en el editor.

utilidades.y × | HackerElite.z × | main.pig ×

```
// Tercer nivel de herencia: usa el poder() de Hacker y solo ca
public class HackerElite extends Hacker {
    public HackerElite(String nombreInicial) {
        super(nombreInicial, 6);
    }

    @Override
    public String rol() {
        return "Hacker de élite";
    }
}

```

Una clase Zetariano: extends, @Override y super.

utilidades.y ✕ | HackerElite.z ✕ | main.pig ✕

```
%funciones

// Recursividad doble: las formas de subir n escalones de uno e
// en dos.
definir fibonacci(entero n) -> entero:
    si(n < 2) entonces
        retornar 1
    retornar fibonacci(n - 1) + fibonacci(n - 2)

// El arreglo llega por referencia desde PigLatin.
definir sumarArreglo([] entero datos, entero cantidad) -> entero
    entero total = 0
    para(entero i = 0; i < cantidad; i++):
        total = total + datos[i]
    retornar total

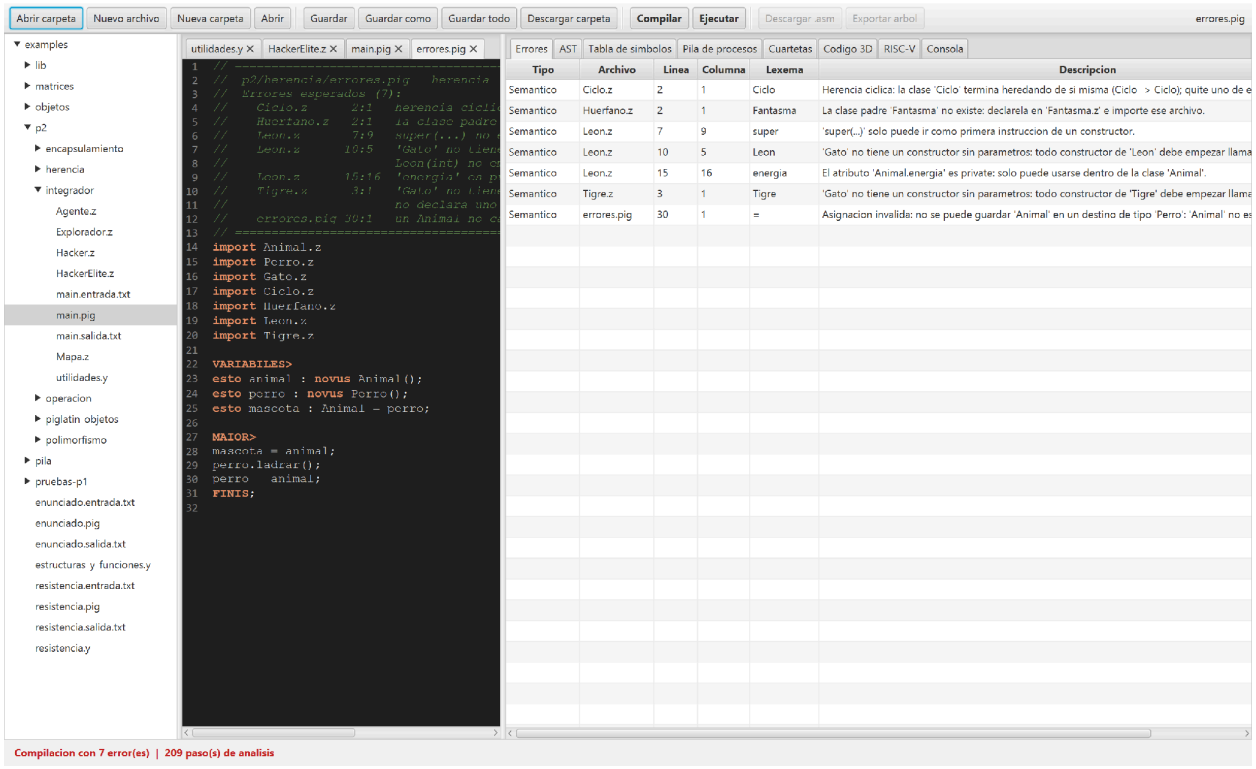
```

Funciones Y? con sus bloques por sangría.

Compilar y leer los errores

1. Abra o seleccione la pestaña del archivo que quiere compilar. Para compilar un programa completo, elija su `.pig`.
2. Pulse "Compilar".
3. Lea la barra de estado. En verde: `Compilacion exitosa`, con la cantidad de símbolos, cuartetas y pasos de análisis. En rojo: `Compilacion con N error(es)`.
4. Si hubo errores, la aplicación abre la pestaña "Errores". Si no, abre la pestaña "RISC-V".

Un `.y` o un `.z` también se puede compilar solo, para revisar sus errores. Solo un `.pig` sin errores se traduce a RISC-V.



Los errores de p2/herencia/errores.pig: cada fila dice el tipo, el archivo, la línea, la columna, el lexema y qué está mal.

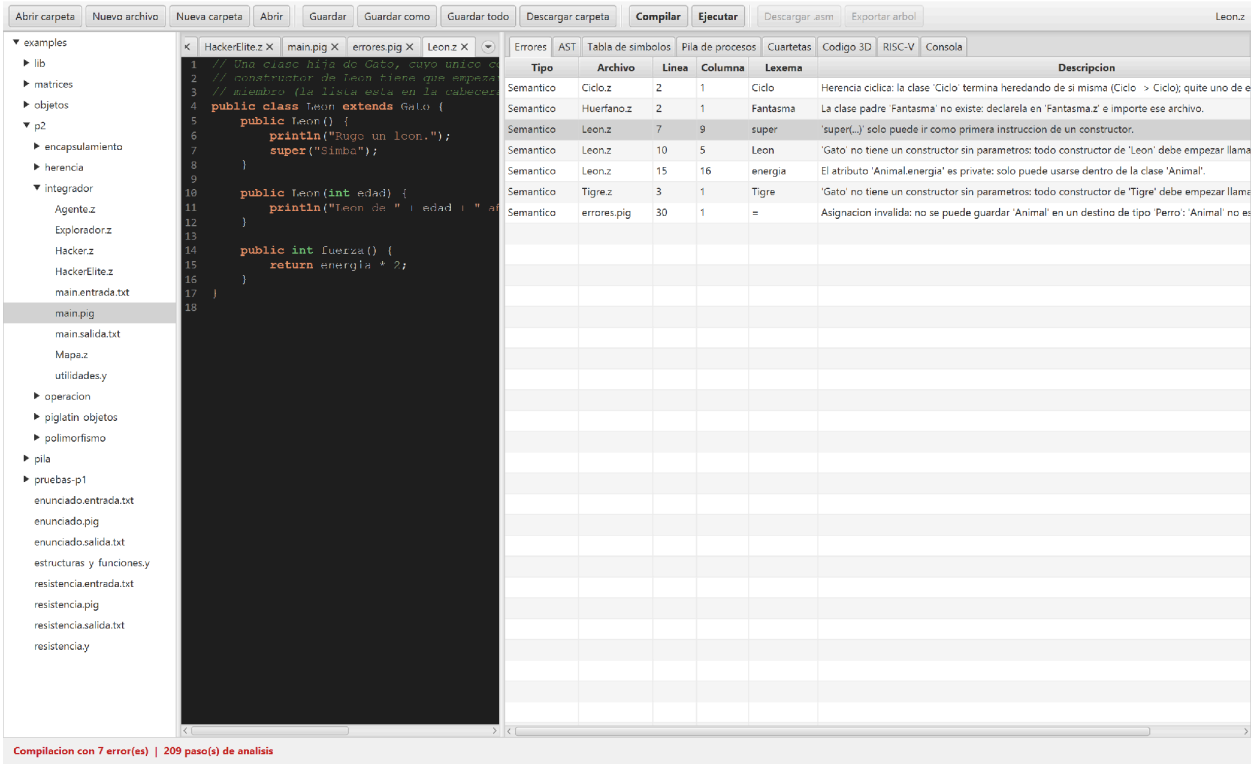
Las columnas de la pestaña Errores.

Columna	Significado
Tipo	`Lexico` (un símbolo que el lenguaje no reconoce), `Sintactico` (algo mal escrito o fuera de lugar) o `Semantico` (está bien escrito pero no tiene sentido, por ejemplo tipos incompatibles).

Columna	Significado
Archivo	El archivo donde está el error, que puede ser uno importado.
Linea y Columna	Dónde empieza el error.
Lexema	El texto que lo provocó.
Descripcion	Qué está mal y, casi siempre, cómo se corrige.

Ir al error

Haga **doble clic** sobre una fila de errores: la aplicación abre el archivo del error (aunque sea un importado y estuviera cerrado) y lleva el cursor a su línea y columna.



Doble clic en el error de Leon.z: se abre Leon.z con el cursor en la línea 7.

Algunos mensajes y qué hacer.

Mensaje	Qué hacer
`El programa termina sin cerrarse: falta 'FINIS;' al final.`	Agregue `FINIS;` como última línea del `.pig`.
`No se encontro el archivo importado 'X'.`	Revise la ruta y las mayúsculas del `import`, relativas a la carpeta del `.pig`.
`La clase 'X' debe estar en un archivo llamado 'X.z'`	Renombre el archivo con "Guardar como" o cambie el nombre de la clase.
`El metodo 'A.m()' sobrescribe a 'B.m()': escriba @Override antes de su declaracion.`	Agregue `@Override` encima del método.

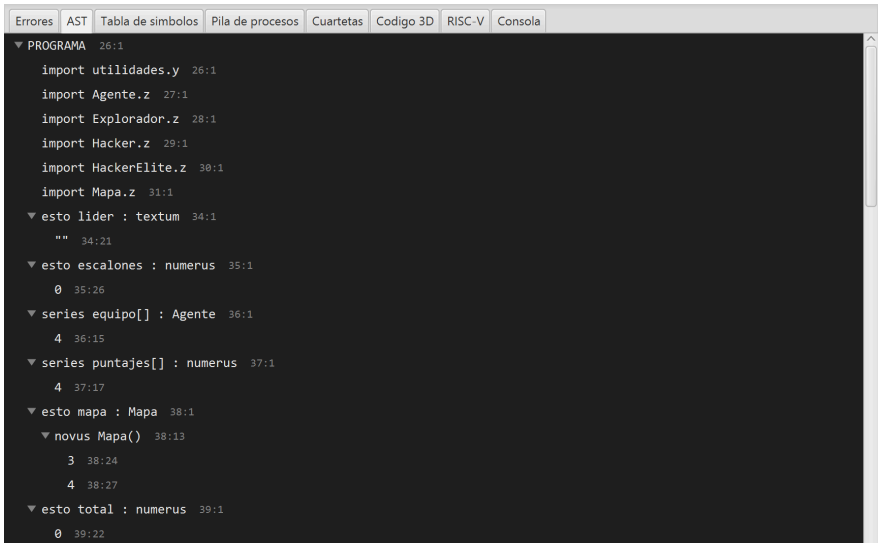
Mensaje	Qué hacer
`El atributo 'Cuenta.pin' es private: ...`	Use un método público de la clase o cambie la visibilidad del atributo.
`El atributo 'Cuenta.movimientos' es default (sin modificador): ...`	Márquelo `protected` si lo usa una clase hija, o `public` si lo usa otra clase o el `.pig`.

Ver los resultados

Después de compilar, las pestañas de la derecha muestran cada etapa de la traducción. Las de un programa con errores quedan vacías, salvo "Errores", "Tabla de simbolos" y "Pila de procesos".

AST

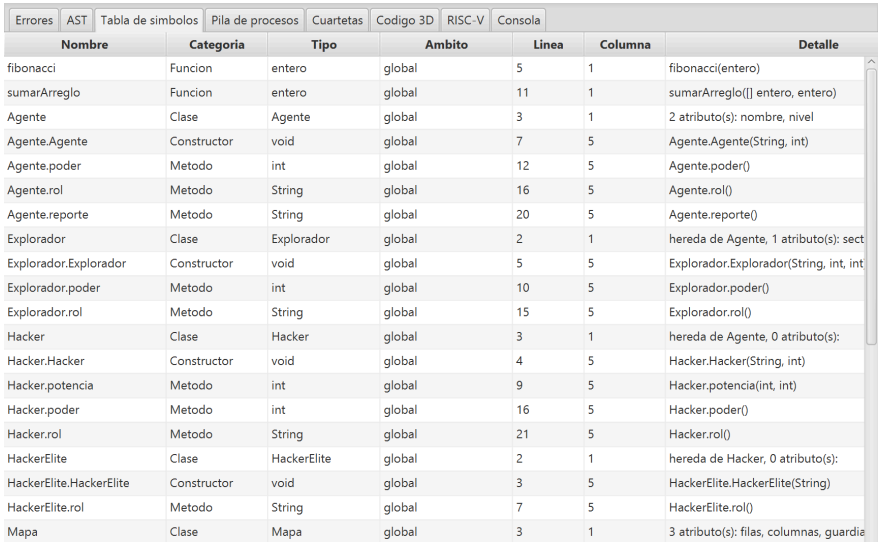
El árbol de sintaxis abstracta del programa. Cada nodo muestra a su derecha la línea y la columna donde está escrito, y con **doble clic** el cursor del editor va hasta él. "Exportar arbol" lo guarda como texto.



La pestaña AST.

Tabla de símbolos

Todo lo que el programa declara: variables, arreglos, funciones, estructuras, clases, atributos, métodos y constructores, con su categoría, su tipo, el ámbito donde vive, su posición y un detalle (la firma de una función, los atributos de una clase o de quién hereda).

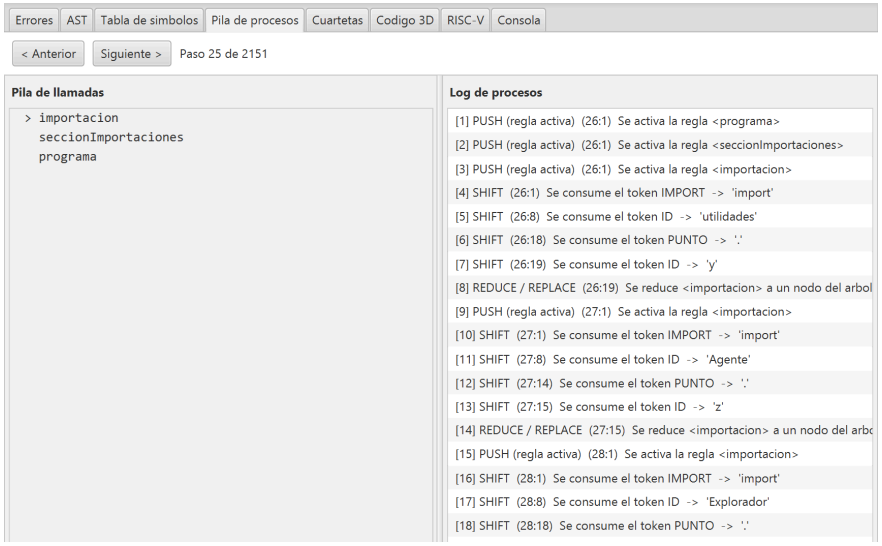


Nombre	Categoria	Tipo	Ambito	Linea	Columna	Detalle
fibonacci	Funcion	entero	global	5	1	fibonacci(entero)
sumarArreglo	Funcion	entero	global	11	1	sumarArreglo([] entero, entero)
Agente	Clase	Agente	global	3	1	2 atributo(s): nombre, nivel
Agente.Agente	Constructor	void	global	7	5	Agente.Agente(String, int)
Agente.poder	Metodo	int	global	12	5	Agente.poder()
Agente.rol	Metodo	String	global	16	5	Agente.rol()
Agente.reporte	Metodo	String	global	20	5	Agente.reporte()
Explorador	Clase	Explorador	global	2	1	hereda de Agente, 1 atributo(s): sect
Explorador.Explorador	Constructor	void	global	5	5	Explorador.Explorador(String, int, int
Explorador.poder	Metodo	int	global	10	5	Explorador.poder()
Explorador.rol	Metodo	String	global	15	5	Explorador.rol()
Hacker	Clase	Hacker	global	3	1	hereda de Agente, 0 atributo(s):
Hacker.Hacker	Constructor	void	global	4	5	Hacker.Hacker(String, int)
Hacker.potencia	Metodo	int	global	9	5	Hacker.potencia(int, int)
Hacker.poder	Metodo	int	global	16	5	Hacker.poder()
Hacker.rol	Metodo	String	global	21	5	Hacker.rol()
HackerElite	Clase	HackerElite	global	2	1	hereda de Hacker, 0 atributo(s):
HackerElite.HackerElite	Constructor	void	global	3	5	HackerElite.HackerElite(String)
HackerElite.rol	Metodo	String	global	7	5	HackerElite.rol()
Mapa	Clase	Mapa	global	3	1	3 atributo(s): filas, columnas, guardia

La pestaña Tabla de simbolos.

Pila de procesos

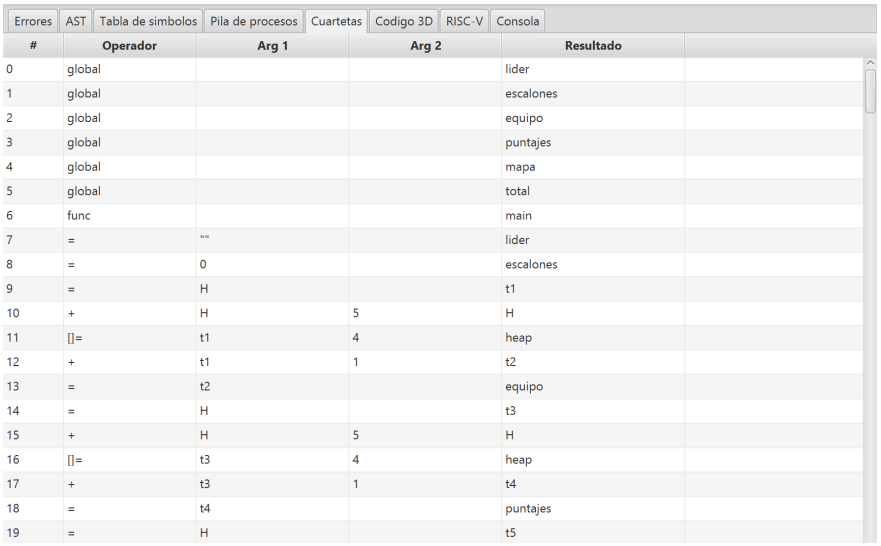
Reproduce paso a paso cómo el analizador reconoció el programa. "< Anterior" y "Siguiente >" avanzan un paso. A la izquierda está la pila de reglas activas en ese paso y a la derecha el registro de lo que se hizo para llegar ahí: PUSH (se activa una regla), SHIFT (se consume un token), REDUCE (se cierra una regla), ACCEPT (programa aceptado) y ERROR.



La pestaña Pila de procesos en el paso 25.

Cuartetas y código 3D

El código intermedio del programa, en dos formas: la tabla de cuartetas (operador, argumento 1, argumento 2 y resultado) y el listado de código de tres direcciones, con sus temporales `t1, t2...` y sus etiquetas `L1, L2...`.



#	Operador	Arg 1	Arg 2	Resultado
0	global			lider
1	global			escalones
2	global			equipo
3	global			puntajes
4	global			mapa
5	global			total
6	func			main
7	=	""		lider
8	=	0		escalones
9	=	H		t1
10	+	H	5	H
11	[]=	t1	4	heap
12	+	t1	1	t2
13	=	t2		equipo
14	=	H		t3
15	+	H	5	H
16	[]=	t3	4	heap
17	+	t3	1	t4
18	=	t4		puntajes
19	=	H		t5

La pestaña Cuartetas.



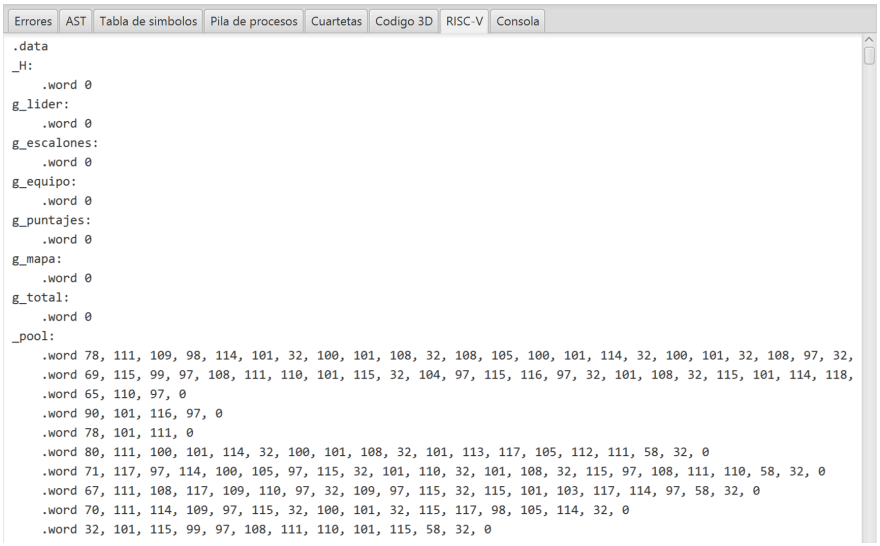

```
global lider
global escalones
global equipo
global puntajes
global mapa
global total

func main():
    lider = ""
    escalones = 0
    t1 = H
    H = H + 5
    heap[t1] = 4
    t2 = t1 + 1
    equipo = t2
    t3 = H
    H = H + 5
    heap[t3] = 4
    t4 = t3 + 1
    puntajes = t4
    t5 = H
    H = H + 4
    heap[t5] = 5
    param t5
    param 3
    param 4
```

La pestaña Codigo 3D.

RISC-V

El programa en ensamblador RISC-V, listo para RARS. Cada bloque lleva encima, como comentario (#), la instrucción de código 3D de la que viene. "Descargar .asm" lo guarda junto al archivo compilado.

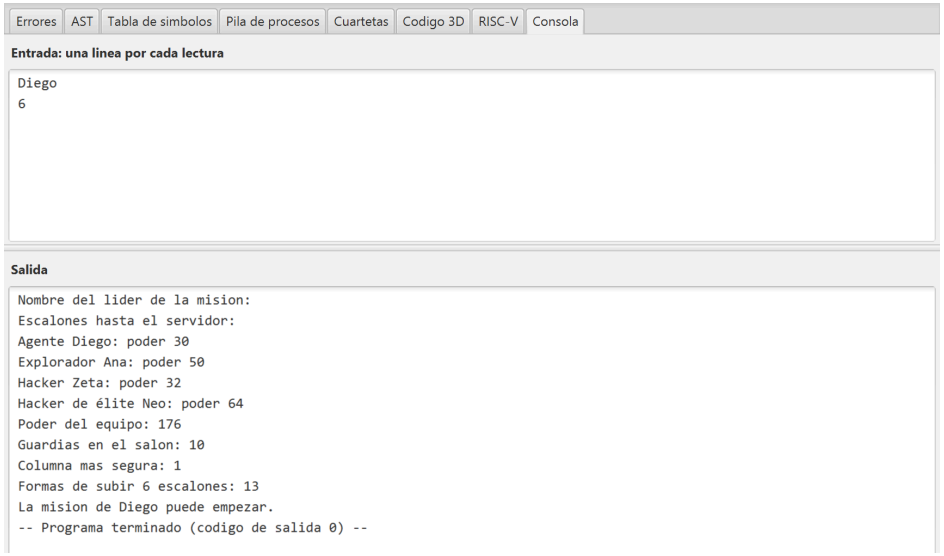


La pestaña RISC-V.

Ejecutar dentro de la aplicación

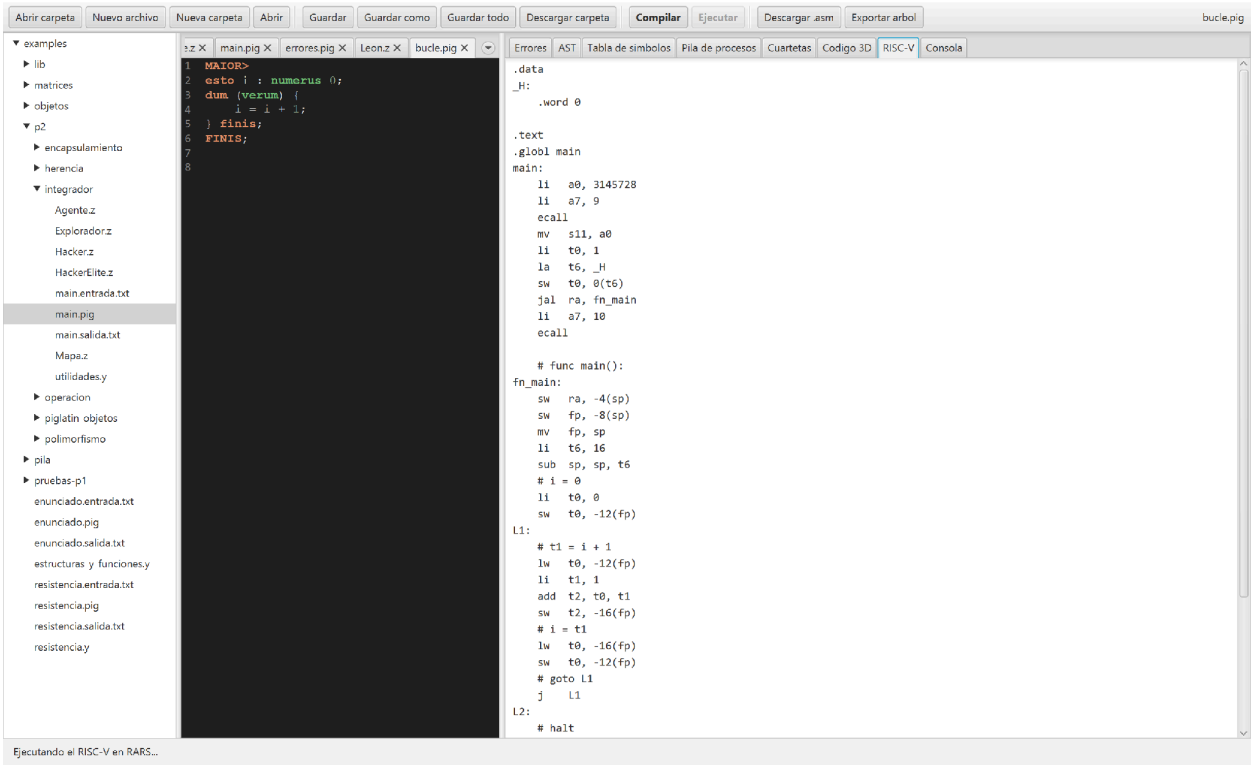
La aplicación ejecuta el ensamblador RISC-V en el simulador RARS, que lleva incluido. No hace falta abrir una terminal ni otro programa.

1. Abra o seleccione el `.pig` que quiere ejecutar.
2. Si el programa lee datos, abra la pestaña "Consola" y escriba en "Entrada" **una línea por cada lectura**, en el orden en que el programa las pide. Por ejemplo, para `p2/integrador` escriba `Diego` en la primera línea y `6` en la segunda.
3. Pulse "Ejecutar". La aplicación compila el archivo y, si no tiene errores, lo ejecuta.
4. Lea la salida en "Salida". La última línea, entre `--`, dice cómo terminó el programa, y la barra de estado lo repite.



La pestaña Consola después de ejecutar p2/integrador.

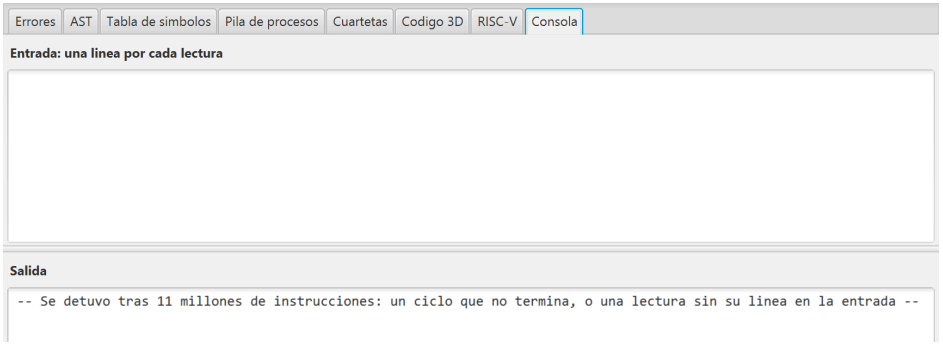
Mientras el programa corre, la barra de estado dice `Ejecutando el RISC-V en RARS...` y el botón "Ejecutar" queda desactivado hasta que termine. Mientras tanto puede seguir usando el resto de la ventana.



Un programa en ejecución: "Ejecutar" está desactivado y la ventana sigue respondiendo.

Cómo puede terminar una ejecución.

Mensaje	**Qué significa**	**Qué hacer**
`Programa terminado (codigo de salida 0)`	El programa terminó bien.	—
`Se detuvo tras 11 millones de instrucciones: ...`	El programa no terminaba: un ciclo infinito, o una lectura sin su línea en "Entrada".	Revise las condiciones de sus ciclos y que "Entrada" tenga una línea para cada lectura.
`Error de ejecucion: ... invalid float input (syscall 6)`	Una lectura de un decimal recibió algo que no es un número.	Escriba un número en esa línea de "Entrada".
`Error de ejecucion: ...`	RARS encontró un error mientras el programa corría.	El texto viene de RARS e indica la instrucción.



Un ciclo infinito: la ejecución se detiene sola y avisa por qué.

Buenas prácticas:

- La entrada se escribe antes de pulsar "Ejecutar": el programa no se detiene a esperar lo que usted escriba.
- Cada lectura toma una línea completa: para leer dos números, escríbalos en dos líneas.
- Lo que escribió en "Entrada" se conserva entre ejecuciones; bórrelo si el próximo programa lee otra cosa.

Comprobar el ensamblador fuera de la aplicación

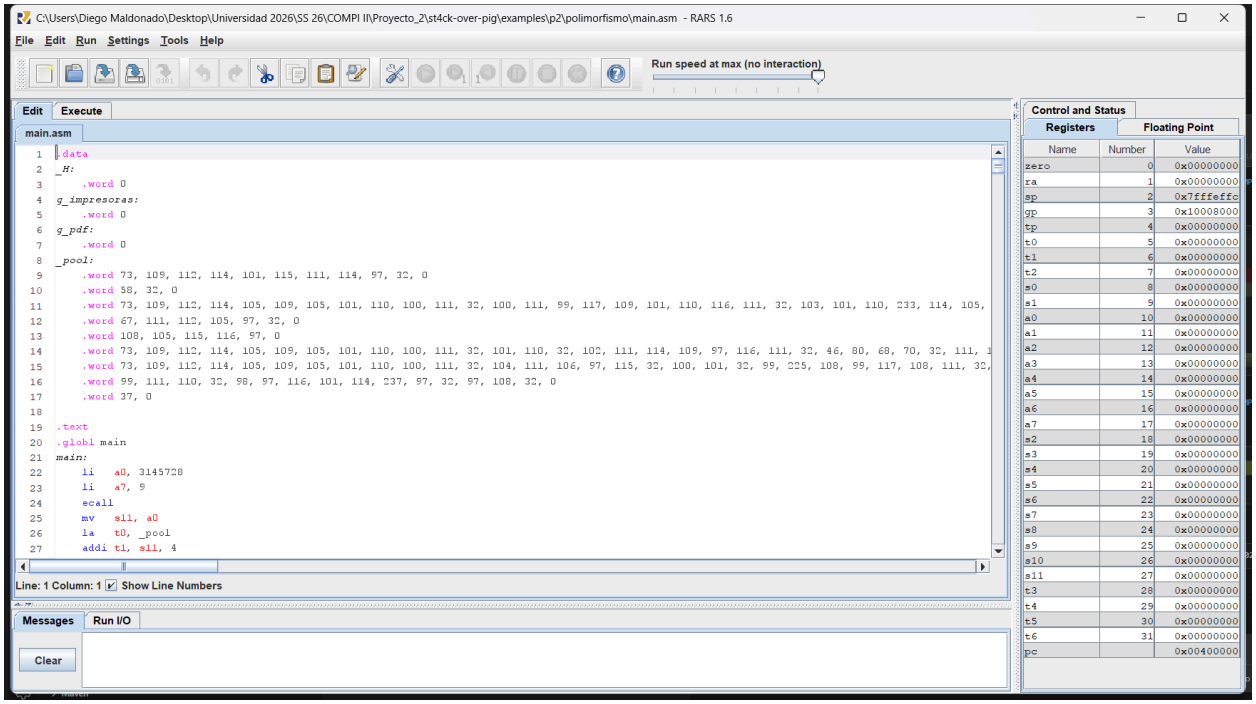
Para comprobar que el ensamblador es real y funciona en otro simulador, ejecútelo en **RARS** por fuera de St4ck Over-Pig. La salida tiene que ser la misma que en la pestaña "Consola".

Descargar el .asm

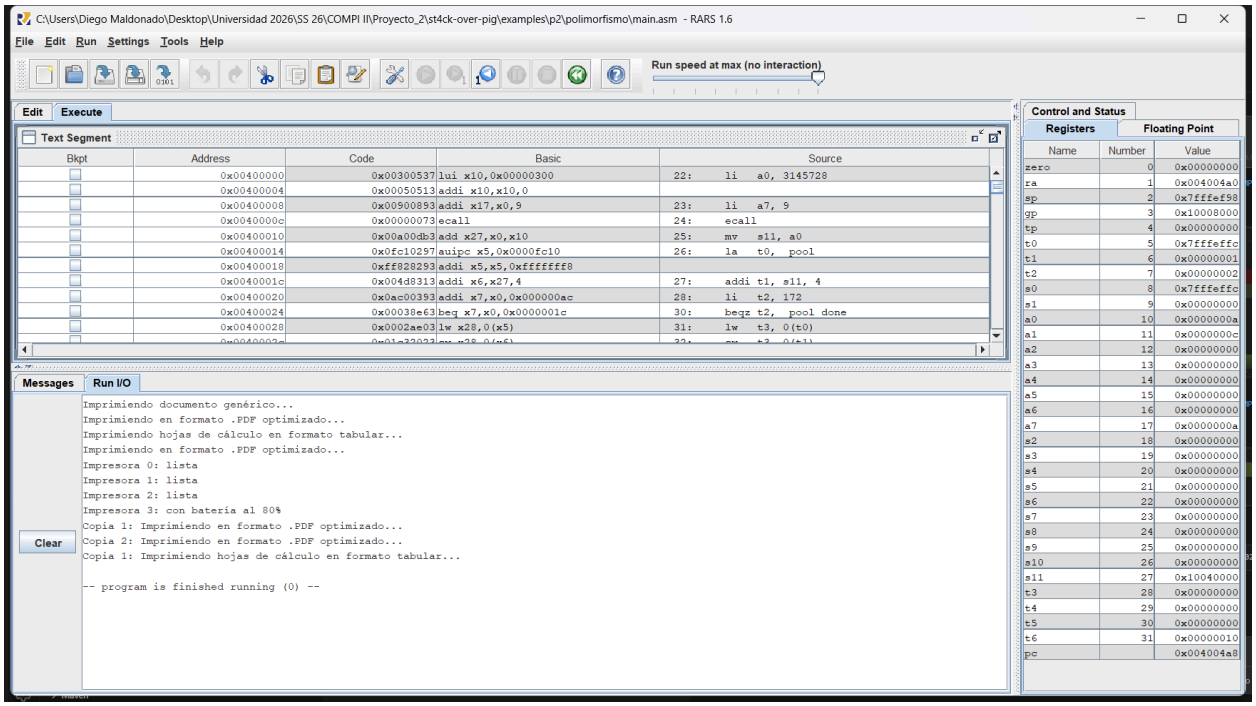
1. Compile el `.pig` sin errores.
2. Pulse "Descargar .asm". El archivo queda junto al `.pig`, con el mismo nombre (`main.asm`), y aparece en el árbol de trabajo.

En la interfaz de RARS

1. Abra RARS: en una terminal dentro de la carpeta del proyecto escriba `java -jar lib/rars/rars/1.6/rars-1.6.jar` (o haga doble clic en ese archivo).
2. Elija *File* → *Open* y abra el `.asm` descargado.
3. Ensámblelo con *Run* → *Assemble* (tecla F3).
4. Ejecútelo con *Run* → *Go* (tecla F5). La salida aparece en el panel *Run I/O*, abajo. Si el programa lee datos, escríbalos ahí cuando los pida.



El .asm de p2/polimorfismo abierto en RARS.



El mismo programa ensamblado y ejecutado: su salida en Run I/O.

En la consola

Para ejecutarlo con un archivo de entrada, use `cmd` (Símbolo del sistema), Git Bash o la terminal de Linux o macOS, dentro de la carpeta del proyecto:

```
java -Dstdout.encoding=UTF-8 -jar lib/rars/rars/1.6/rars-1.6.jar nc me sm se1 ae2
    examples/p2/integrador/main.asm < examples/p2/integrador/main.entrada.txt
```

PowerShell no admite `<` y además parte en dos la opción `-Dstdout.encoding=UTF-8`. Desde PowerShell, deje que `cmd` corra el mismo comando:

```
cmd /c "java -Dstdout.encoding=UTF-8 -jar lib/rars/rars/1.6/rars-1.6.jar nc me sm se1 ae2
    examples/p2/integrador/main.asm < examples/p2/integrador/main.entrada.txt"
```

La salida coincide con la de la pestaña "Consola" y con `main.salida.txt`, salvo la línea final `-- ... --`, que es de la aplicación.

Ejemplos incluidos

La carpeta `examples` tiene programas listos para probar. Junto a cada programa principal está lo que se escribe en la consola (`<programa>.entrada.txt`, solo si lee algo) y lo que debe imprimir (`<programa>.salida.txt`). Los `errores.pig` no compilan a propósito: su cabecera lista los errores que dan.

Los ejemplos.

Ejemplo	Lenguajes	Qué demuestra
`enunciado.pig` y `lib/Funciones.y`	PigLatin, Y?	Los fragmentos del enunciado de los dos lenguajes: declaraciones, arreglos, estructuras, funciones y ciclos. Lee de la consola.
`estructuras_y_funciones.y`	Y?	Las estructuras y funciones del enunciado de Y?, analizadas solas: los bloques por sangría.
`resistencia.pig` y `resistencia.y`	PigLatin, Y?	Funciones, recursividad, arreglos, estructuras, los tres ciclos, `interrumpe`/`perge` y `elegir`. Lee un número.
`matrices/`	PigLatin, Y?	Matrices con sus valores fila por fila, aplanadas en el heap.
`objetos/`	PigLatin, Zetariano	Los fragmentos del enunciado de Zetariano: expresiones, arreglos, matrices, objetos, `null`, `switch` con fallthrough, los ciclos, `readln` y recursividad. Lee un texto y un número.
`pila/`	Los tres	La prueba de la auxiliar: una pila con `Nodo.z` y `Pila.z`, manejada desde un menú.
`pruebas-p1/00_hello_world/`	Los tres	Hola mundo desde los tres lenguajes, y `Minimo.pig`.
`pruebas-p1/01_calculadora/`	PigLatin, Y?	Arreglos globales por referencia, un arreglo dentro de una estructura y recursividad.
`pruebas-p1/02_lista_doble/`	Los tres	Una lista doblemente enlazada: referencias encadenadas, `null`, sobrecarga y `elegir :`.
`pruebas-p1/03_laberinto/`	Los tres	Matrices como atributo con dimensiones variables, recursividad (flood fill) y `switch` con fallthrough.
`pruebas-p1/04_ahorcado/`	Los tres	Arreglos `char[]` y `boolean[]` creados con `new` en el constructor, `elegir :` y el ternario.
`p2/operacion/`	PigLatin, Zetariano	El ejemplo de clase y del parcial: el método recursivo `sumar`. El manual técnico lo sigue paso a paso.
`p2/encapsulamiento/`	PigLatin, Zetariano	`public`, `protected`, default y `private` en atributos, métodos y constructores; default solo dentro de su clase. `errores.pig`: los 27 usos prohibidos.
`p2/herencia/`	PigLatin, Zetariano	`Animal` y `Perro` del enunciado: lo heredado, `super(...)`, el `super()` implícito y tres niveles de herencia. `errores.pig`: 7 errores de herencia.
`p2/polimorfismo/`	PigLatin, Zetariano	`Impresora`, `ImpresoraPDF` e `ImpresoraExcel` del enunciado en un arreglo: cada `imprimir()` responde por la clase del objeto. `errores.pig`: las reglas de `@Override`.
`p2/piglatin-objetos/`	PigLatin, Zetariano	`novus Profesor(..) : Persona` y `null : Persona` del enunciado. `errores.pig`: 4 errores de tipos de objeto.

Ejemplo	Lenguajes	Qué demuestra
`p2/integrador/`	Los tres	La misión de la resistencia: herencia en tres niveles, un arreglo de objetos, una matriz, recursividad en Y? y en Zetariano, y un arreglo que llega a Y? por referencia. Lee un nombre y un número.
`p2/banco/`	Los tres	El banco de pruebas de la auxiliar, con `movimientos` declarado `protected`: herencia, `@Override`, despacho sobre un arreglo de `Cuenta`, `null`, `double` con `int` y una estructura de Y? por referencia.

Probar un ejemplo

1. Pulse "Abrir carpeta" y elija `examples/p2/integrador`.
2. Haga doble clic en `main.entrada.txt` y copie su contenido.
3. Abra `main.pig`, vaya a la pestaña "Consola" y pegue el contenido en "Entrada".
4. Pulse "Ejecutar" y compare la salida con `main.salida.txt`.

Cuidados y buenas prácticas

- Trabaje dentro de una carpeta de trabajo: así los `import` se resuelven y el árbol muestra todos sus archivos.
- Guarde a menudo ("Guardar todo"). Antes de entregar o de cambiar de computadora, use "Descargar carpeta" para tener una copia.
- Mantenga cada clase de Zetariano en su propio archivo con el mismo nombre que la clase.
- Los archivos se guardan en UTF-8. Un archivo hecho en otro editor con otra codificación también se abre.
- Si una ejecución tarda mucho, probablemente un ciclo no termina: la aplicación la detiene sola tras 11 millones de instrucciones.
- Para que la salida de RARS por consola muestre bien los acentos, deje `-Dstdout.encoding=UTF-8` en el comando.

Problemas frecuentes

Problemas frecuentes y su solución.

Problema	Solución
Pulso "Ejecutar" en un `.y` o un `.z` y no se ejecuta nada.	Solo un `.pig` tiene programa principal (`MAIOR>`). Abra el `.pig` que importa ese archivo y ejecútelo.
"Descargar .asm" está desactivado.	Compile primero un `.pig` sin errores.
La ejecución dice que se detuvo tras 11 millones de instrucciones.	Revise que sus ciclos terminen y que "Entrada" tenga una línea para cada lectura.
`Guarde el archivo para poder resolver sus importaciones.`	Guarde el `.pig` en la carpeta donde están los archivos que importa.
Una clase compila sola, pero el `.pig` dice `La clase 'Nodo' no existe.`	Agregue al `.pig` el `import` de esa clase, aunque solo la use otra clase.
Un número que leí sale como 0.	La línea de "Entrada" no era un número, o faltaba esa línea.
La ventana no cabe en la pantalla.	La aplicación se ajusta sola al abrir; también puede maximizarla.
`mvn` o `java` "no se reconoce como comando".	Instale el JDK 21 y Maven, y agréguelos al `PATH`.
La primera vez `mvn clean javafx:run` tarda mucho.	Es normal: Maven descarga sus dependencias. Las veces siguientes es más rápido.

Glosario

Términos usados en la aplicación.

Término	Significado
AST	Árbol de sintaxis abstracta: la estructura del programa que arma el analizador.
C3D	Código de tres direcciones: instrucciones simples con a lo sumo dos operandos y un resultado.
Cuarteta	Una instrucción de C3D escrita como (operador, argumento 1, argumento 2, resultado).
RISC-V	Arquitectura de procesador; su lenguaje ensamblador es la salida final del compilador.
RARS	Simulador de RISC-V que ejecuta el ensamblador, dentro y fuera de la aplicación.
Heap	Memoria donde viven las cadenas, los arreglos, las estructuras y los objetos.
Pila (stack)	Memoria donde vive cada llamada a una función: sus parámetros, variables y temporales.
Error léxico, sintáctico y semántico	Un símbolo no reconocido; algo mal escrito o fuera de lugar; algo bien escrito que no tiene sentido.